

Backend Mimari Desenleri

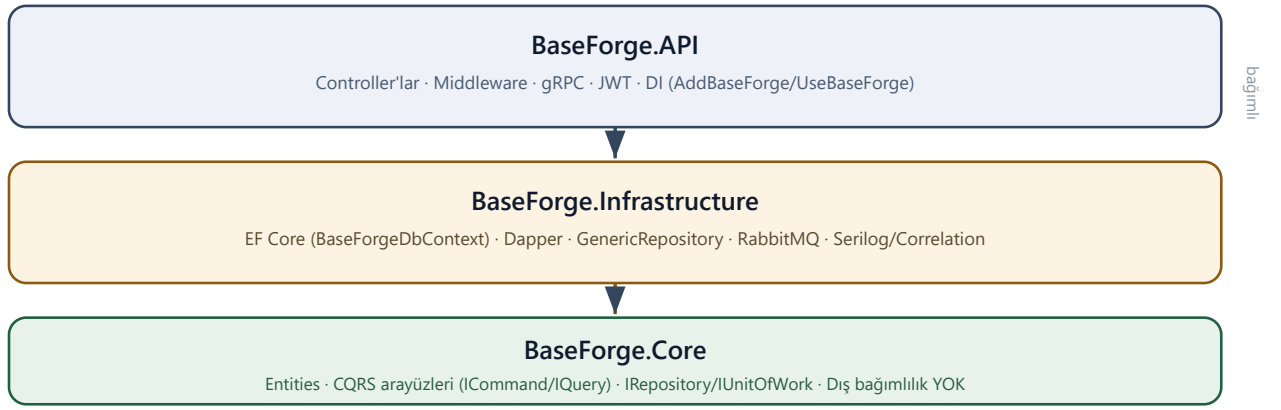
BaseForge .NET 10 base kütüphanesinin backend tarafında kullanılan mimari desenlerin (Clean Architecture, CQRS, Repository/UnitOfWork, Transactional Outbox, Inbox/Idempotency, RabbitMQ Pub/Sub + DLQ, Merkezi Loglama, gRPC, JWT, Multi-Tenancy) ne olduğu, neden seçildiği ve nasıl kullanıldığının diyagramlarla anlatımı.

İçindekiler

1. Genel Mimari — Clean Architecture Katmanları
2. CQRS — MediatR Üzerine
3. Repository + Unit of Work Deseni
4. BaseEntity: Audit, Soft Delete, Multi-Tenancy
5. Transactional Outbox Pattern
6. Inbox Pattern (Tüketici Idempotency'si)
7. RabbitMQ Pub/Sub, Dead-Letter Queue ve Kanal Havuzu
8. Merkezi Loglama ve Correlation ID Yayılımı
9. gRPC — Senkron Servisler Arası İletişim
10. Kimlik Doğrulama (JWT)
11. Docker Altyapı Topolojisi

1. Clean Architecture Katmanları

BaseForge, **Opinionated Library** yaklaşımıyla üç NuGet paketine bölünmüş Clean Architecture katmanlaması kullanır. Her katmanın bağımlılık yönü tek yönlüdür: **Core** hiçbir dış bağımlılık almaz (yalnızca MediatR arayüzleri), **Infrastructure** yalnızca **Core** 'a bağımlıdır, **API** ise ikisine birden bağımlı olabilir. Bu sıralama sayesinde iş kuralları (Core) altyapı detaylarından (EF Core, RabbitMQ, ASP.NET Core) tamamen izole kalır — örneğin veritabanı sağlayıcısı değişse bile Core hiç etkilenmez.



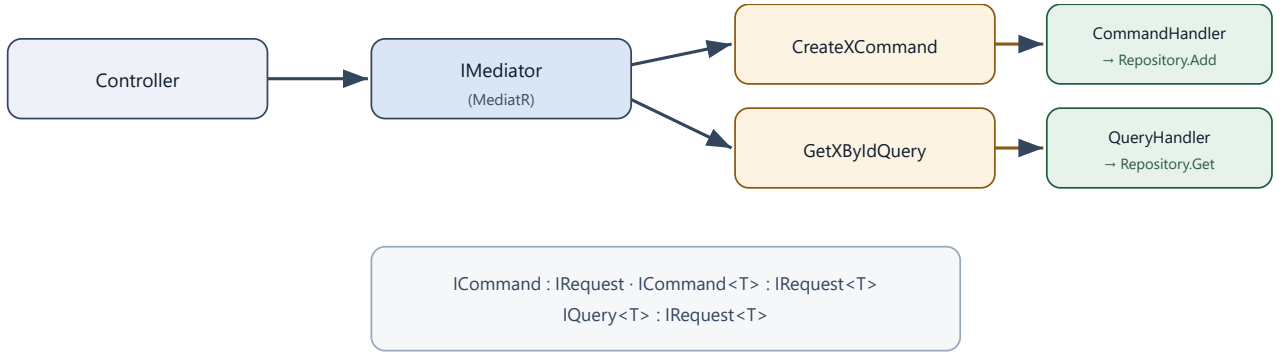
Şekil 1 — Katman bağımlılık yönü: API → Infrastructure → Core (tek yönlü, geri dönüş yok).

Kullanım

Yeni bir mikroservis, üç paketi de referans alıp `builder.Services.AddBaseForge(options => { ... })` ile tek satırda kayıt olur; `app.UseBaseForge()` ile de gerekli middleware zinciri (correlation id → exception handling → request logging → auth) otomatik kurulur.

2. CQRS — MediatR Üzerine

BaseForge sıfırdan bir CQRS altyapısı yazmaz; `ICommand / IQuery / ICommandHandler / IQueryHandler` (Core/CQRS), MediatR'in `IRequest / IRequestHandler`'i üzerine ince, isimlendirici (marker) sarmalayıcılardır. Amaç: MediatR'in kanıtlanmış dispatch mekanizmasını kullanırken, komut/sorgu ayrımını (CQS) kod tabanında adlandırma ile açıkça görünür kılmak.



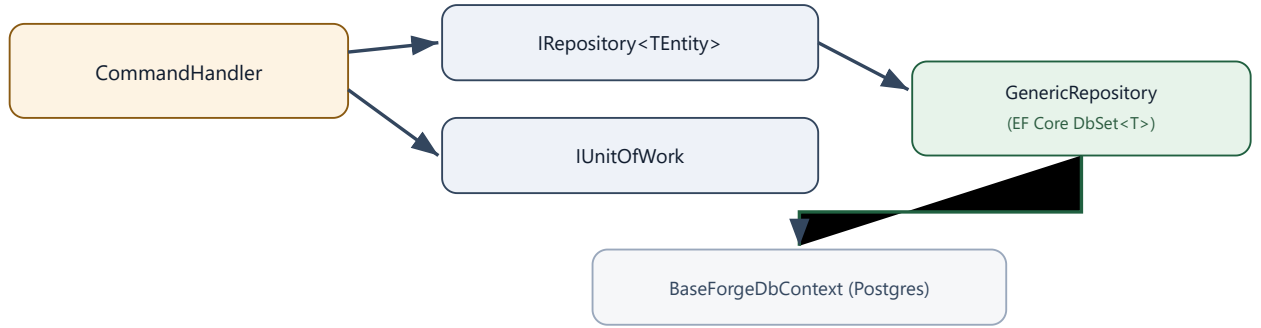
Şekil 2 — Controller, MediatR üzerinden Command/Query'yi ilgili handler'a yönlendirir; CodeGen bu üçlüyü her entity için otomatik üretir.

Kullanım

CodeGen her entity için `Create/Update/Delete{Entity}Command` ve `Get{Entity}ByIdQuery / List{Entity}Query` üretir; controller'lar bu komut/sorguları `IMediator.Send( ... )` ile tetikler, hiçbir zaman doğrudan repository'ye dokunmaz. `options.EnableCQRS(typeof(Program).Assembly)` ile handler'lar taranıp DI'a eklenir.

3. Repository + Unit of Work Deseni

`IRepository<TEntity, TKey>` (Core) — `GetByIdAsync`, `ListAllAsync`, `ListPagedAsync(skip, take, sortBy, filter?, ct)`, `AddAsync`, `UpdateAsync`, `DeleteAsync` — generic bir CRUD sözleşmesi sunar; `IUnitOfWork` ise `SaveChangesAsync` / `BeginTransactionAsync` / `CommitAsync` / `RollbackAsync` ile explicit transaction yönetimini soyutlar. Somut implementasyon `GenericRepository` (Infrastructure) EF Core üzerine kuruludur; sayfalama sorgularında `sortBy` alanı **Dynamic LINQ** ile çözümlenir (geçersiz alan adı, sessizce yok sayılmak yerine `ValidationException` olarak "fail loud" döner).



Şekil 3 — Handler, Core arayüzleri üzerinden çalışır; somut EF Core detayı yalnızca Infrastructure'da bilinir.

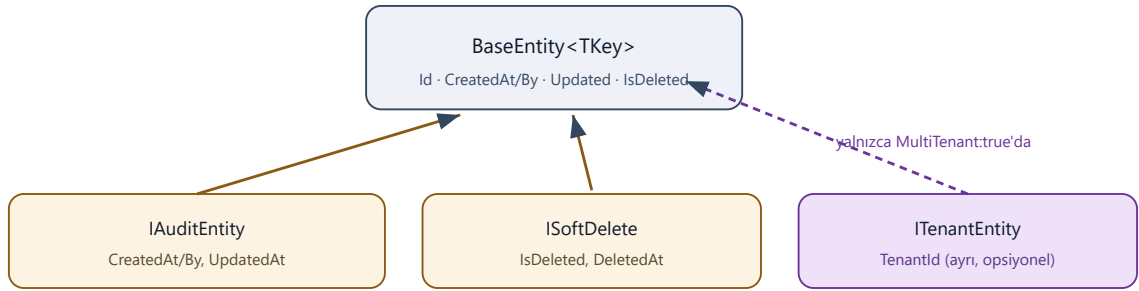
Kullanım

Ağır okuma/karmaşık join gerektiren sorgularda Dapper devreye girer — aynı `DbConnection` ve transaction'ı EF Core ile paylaşır, sonuç doğrudan DTO'ya map edilir. Yazma/tracking/migration işlerinin tamamı EF Core'da kalır (PDF spesifikasyonundaki "ADO.NET, ORM yok" kararı, proje ilerledikçe EF Core + Dapper'a revize edildi).

4. BaseEntity: Audit, Soft Delete, Multi-Tenancy

`BaseEntity<TKey>` : `IAuditEntity`, `ISoftDelete` her entity'ye `Id`, `CreatedAt`, `UpdatedAt`, `CreatedBy`, `IsDeleted`, `DeletedAt` alanlarını otomatik kazandırır; `BaseForgeDbContext.SaveChanges*` her çağrıda bu alanları change tracker üzerinden doldurur, `Delete` çağrılarını fiziksel silme yerine `IsDeleted = true` 'ya çevirir ve silinmiş kayıtları global query filter ile gizler.

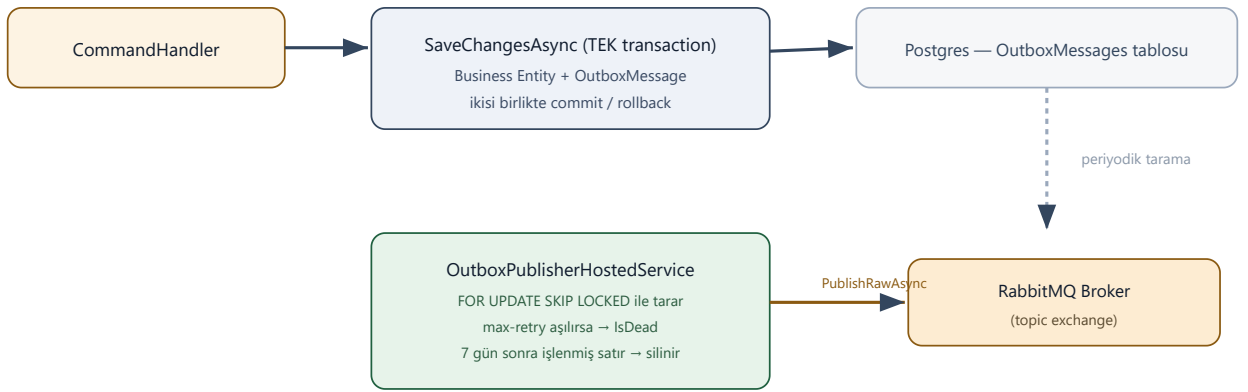
`ITenantEntity` (yalnızca `TenantId`) ise **ayrı bir marker interface'tir** — `BaseEntity` 'ye gömülmez (mevcut tüm servisleri etkileyen breaking change olmasın diye); yalnızca `MultiTenant: true` işaretli servislerin entity'lerine CodeGen tarafından eklenir. EF Core her entity tipi için tek bir query filter'a izin verdiğinden, soft-delete ve tenant filtreleri `Expression.AndAlso` ile **tek bir birleşik filtrede** kurulur.



Şekil 4 — Audit + Soft-Delete taban sınıfta zorunlu; Multi-Tenancy ayrı, opsiyonel bir marker.

5. Transactional Outbox Pattern

Çözdüğü sorun: Bir command handler önce DB'ye commit edip sonra RabbitMQ'ya publish etseydi, commit başarılı olur ama publish (ağ/broker hatası) başarısız olursa business değişiklik kalıcı olur ama event hiç yayınlanmazdı — sessiz veri/olay kaybı. Outbox bunu ortadan kaldırır: event, business entity ile **aynı** `SaveChangesAsync` çağrısında, aynı transaction'da bir `OutboxMessage` satırı olarak yazılır. Gerçek broker'a gönderim, ayrı bir arka plan relay'i (`OutboxPublisherHostedService`) tarafından, periyodik olarak (varsayılan 2sn) yapılır.

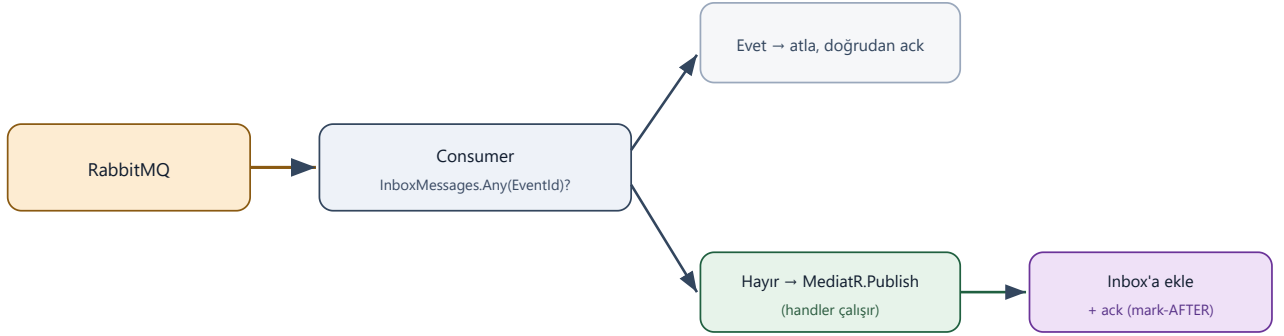


Şekil 5 — Publish, commit'ten SONRA değil; commit'in İÇİNDE (aynı transaction) gerçekleşir. Gerçek broker gönderimi ayrı, güvenilir bir relay'e devredilir.

Bilinen kısıtlar: Teslim garantisi *at-least-once*'tur (exactly-once değil) — relay publish'ten sonra **ProcessedAt** commit edilmeden çökerse mesaj bir sonraki taramada tekrar gönderilebilir. Bu, Bölüm 6'daki Inbox pattern ile tüketici tarafında telafi edilir.

6. Inbox Pattern (Tüketici Idempotency'si)

Outbox'un at-least-once garantisi, aynı olayın tüketici tarafında iki kez işlenmesi ihtimalini doğurur. Inbox pattern bunu kapatır: tüketici, MediatR'a dağıtmadan ÖNCE `InboxMessages` tablosunda aynı `EventId`'yi arar; bulursa handler'ı tekrar çalıştırmadan mesajı "ack"ler. Bulamazsa handler çalışır, **başarıyla tamamlandıktan SONRA** (mark-after) Inbox satırı yazılır ve ardından ack gönderilir.

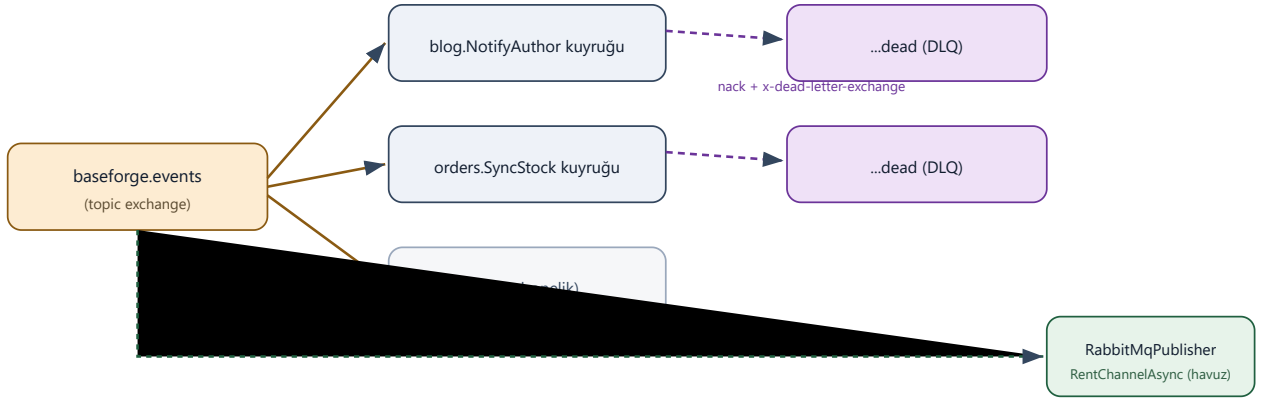


Şekil 6 — Mark-AFTER sırası kasıtlıdır: handler çökerse Inbox satırı hiç yazılmaz, mesaj güvenle tekrar denenir.

Neden mark-before değil: İşaretleme handler'dan ÖNCE yapılıyorsa, handler çökünce (ack hiç gitmez, mesaj yeniden teslim edilir) satır zaten commit edilmiş olacağından event yanlışlıkla "zaten işlendi" sayılıp **tamamen kaybolurdu**. Kalan ince kısıt: handler'ın kendi DB etkileri ile Inbox işaretleme aynı transaction'da değildir — nadir bir gerçek duplicate riski dokümente edilmiştir.

7. RabbitMQ Pub/Sub, Dead-Letter Queue ve Kanal Havuzu

Tüm servisler tek bir paylaşılan **topic exchange** (`baseforge.events`) kullanır; routing key olayın `EventType` 'ından türetilir (`servis/EntityKind` → `servis.EntityKind`). Her abonelik kendi dayanıklı kuyruğuna sahiptir. Sürekli başarısız olan bir mesaj artık sessizce kaybolmaz: her kuyruk bir **dead-letter exchange**'e (`.dlx` , fanout) bağlıdır, reddedilen mesaj `{queue}.dead` kuyruğuna yönlendirilir.



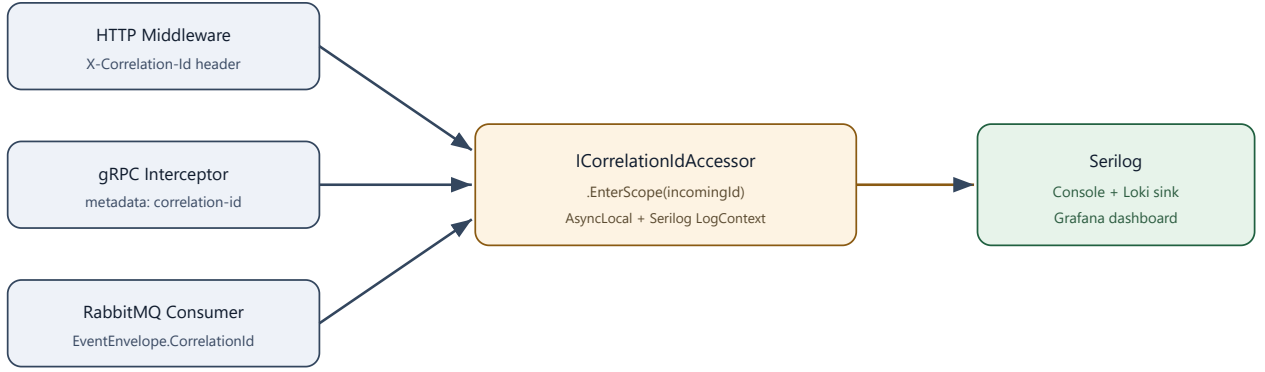
Şekil 7 — Tek paylaşılan exchange, her abonelik kendi kuyruğu + kendi dead-letter kuyruğu; publisher kanalları havuzdan kiralar.

Kanal havuzu: `RabbitMqConnectionManager`, tek bir `IConnection` 'ı paylaşır ama kısa ömürlü çağırımlar (publisher) için sınırlı (kapasite 10) bir `RentChannelAsync` / `ReturnChannelAsync` havuzu sağlar — her publish'te aç/kapat yerine kanal yeniden kullanılır. Tüketicinin kendi kanalı zaten uygulama ömrü boyunca tek olduğu için havuza ihtiyaç duymaz.

Bilinçli kapsam dışı: Otomatik, gecikmeli yeniden-deneme (retry-with-delay) eklenmedi — bu, TTL+DLX zincirleme (delay-queue pattern) gerektirir; canlı bir broker'a karşı doğrulanmadan doğru kurulduğuna güvenmek riskli görüldü.

8. Merkezi Loglama ve Correlation ID Yayılımı

Her servis Serilog ile yapılandırılmış (structured) log üretir; `Serilog:LokiUrl` appsettings anahtarı doluysa aynı zamanda **Grafana Loki**'ye push edilir (boşsa yalnızca konsola yazılır — Loki'nin ayakta olması bir ön koşul değildir). Bir isteğin HTTP → gRPC → RabbitMQ zinciri boyunca aynı `CorrelationId` ile takip edilebilmesi için `ICorrelationIdAccessor` (`AsyncLocal` tabanlı) üç sınırdan da paylaşılır.



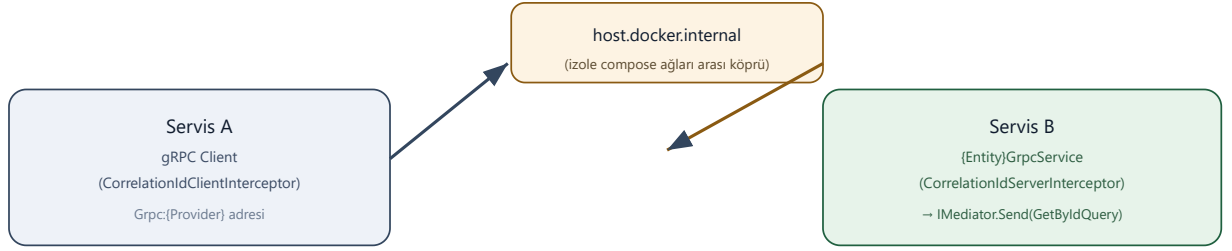
Şekil 8 — Üç farklı sınır (HTTP/gRPC/RabbitMQ), aynı paylaşılan `EnterScope` yardımcısı üzerinden tek bir mekanizmaya bağlanır.

Kullanım

`builder.AddBaseForgeLogging("servis-adı")` tek satırda Serilog'u yapılandırır; Loki'ye yazılamazsa `SelfLog` ile stderr'e tanılama düşer. Loki'nin retention politikası 7 gün olarak yapılandırıldı (`grafana/loki-config.yaml`); Grafana'daki hazır dashboard `Service / CorrelationId` filtresiyle logları gösterir.

9. gRPC — Senkron Servisler Arası İletişim

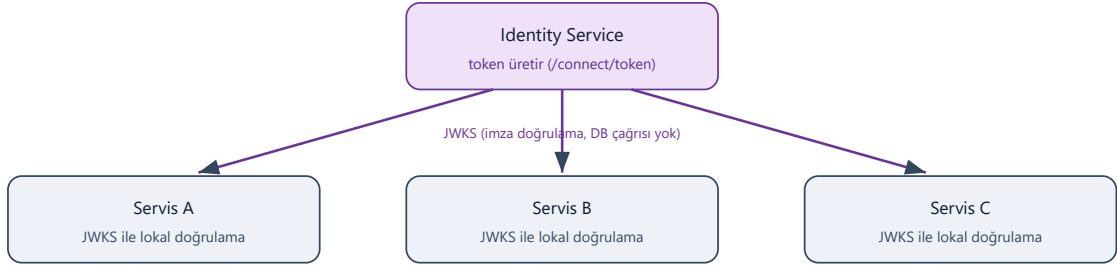
Her servis, kendi entity'lerini otomatik olarak bir gRPC servisi üzerinden dışarı açar (opt-in/opt-out yok — sıralama bağımlılığından kaçınmak için her entity her zaman expose edilir); sunucu tarafı, mevcut CQRS `Get{Entity}ByIdQuery`'sini MediatR ile çağırır — veri erişimi hiç tekrarlanmaz. Kestrel her serviste iki port dinler: `Http` (REST/Scalar) ve `Grpc` (h2c, TLS'siz HTTP/2) — çünkü Kestrel tek portta TLS olmadan HTTP/1.1 ile HTTP/2'yi otomatik ayırt edemez.



Şekil 9 — Kardeş-spec zengin çözümleme ile tipli client üretilir; bulunamazsa yalnızca Id'li minimal bir stub'a düşülür.

10. Kimlik Doğrulama (JWT)

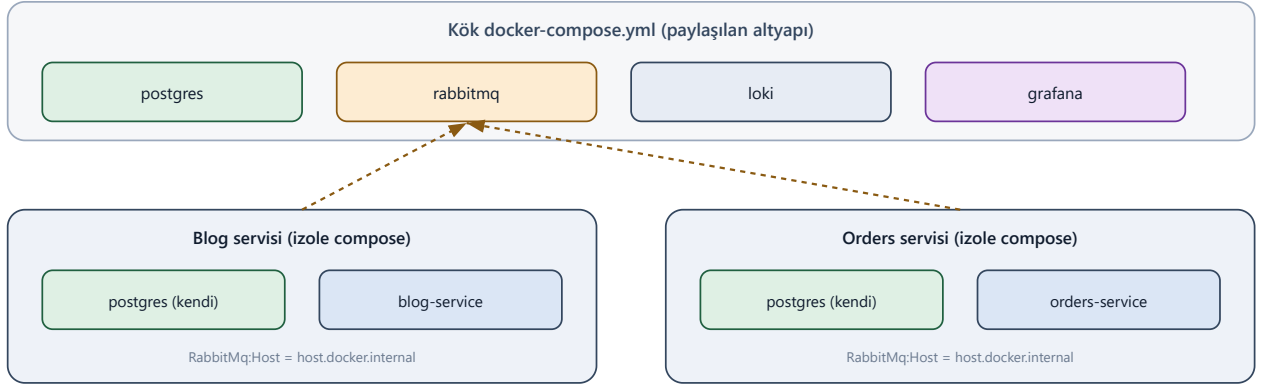
Merkezi tek bir **Identity Service** (OpenIddict + ASP.NET Identity) JWT/OAuth2 token üretir. Her servis, token'ı **kendi middleware'inde lokal olarak** doğrular — her istekte merkezi bir DB çağrısı yapılmaz. İki mod desteklenir: **Authority** verilirse asimetrik/JWKS (imza, Identity'nin discovery/JWKS ucundan otomatik çözülür — önerilen); verilmezse **SigningKey** ile simetrik HMAC.



Şekil 10 — Token üretimi merkezi, doğrulama dağıtık — her servis kendi başına, ağ çağrısı olmadan doğrular.

11. Docker Altyapı Topolojisi

Kök `docker-compose.yml`, **paylaşılan altyapıyı** barındırır: `postgres` (tek-örnek geliştirme için), `rabbitmq`, `loki`, `grafana`. Üretilen her mikroservis ise kendi **izole** docker-compose'una ve kendi Postgres container'ına sahiptir (Database-per-Service) — ama RabbitMQ/Loki için kendi container'ını açmaz, paylaşılan olana `host.docker.internal` üzerinden bağlanır.



Şekil 11 — Database-per-Service (her serviste kendi Postgres'i) + paylaşılan mesajlaşma/gözlemlenebilirlik altyapısı.